

Clothing & Hair Brand

E-Commerce Build Handover — powered by the Pulsify stack

This document explains how to build a new online store for clothing and hair products reusing the exact same technology stack and accounts behind Pulsify (pulsefy.co.za): **Vercel + Supabase + GitHub + a single-file front end**. It covers account reuse, the data model, the page structure, payments, and a phased build plan you can hand to a fresh Claude Code session.

Field	Value
Project type	E-commerce store (apparel + hair / beauty products)
Stack	Vercel (hosting + serverless API) + Supabase (DB/Auth/Storage)
Front end	Single-page vanilla JS/HTML/CSS (same pattern as Pulsify)
Payments	Paystack (reuse your existing integration code)
Email	Resend (reuse api/email.js pattern)
Prepared for	Thabiso Collins Engane
Date	2026-06-23

1. Can I reuse my existing accounts?

Yes — for all three. You do **not** need new credentials. The golden rule: **new project / new repo, same login.**

Service	Reuse the account?	What you create
Supabase	Yes	A brand-new PROJECT under your org. Separate DB, keys, auth users, storage.
Vercel	Yes	A new PROJECT, import the new GitHub repo. Add its own env vars.
GitHub	Yes	A new REPO (e.g. 'aura-store'). Keeps code separate from Pulsify.
Paystack	Yes	Same merchant account; optionally a sub-account for clean reporting.
Resend	Yes	Same account; verify the new store's sending domain if different.

Why never share one Supabase project across two apps

- The database tables would collide (Pulsify 'events' vs store 'products').
- Auth users would mix — a Pulsify signup would appear in your store.
- Storage buckets and RLS policies would fight each other.
- A separate project keeps billing, backups and security cleanly scoped.

Supabase free tier allows 2 active projects. If Pulsify already uses one, the store is your second — still free. Beyond that you upgrade or pause an idle project.

2. Architecture (same shape as Pulsify)

The store mirrors Pulsify's proven layout so you can lift patterns directly:

Layer	Pulsify	Clothing/Hair Store
Hosting	Vercel	Vercel (same)
API	api/*.js serverless fns	api/*.js serverless fns (same)
Database	Supabase Postgres	Supabase Postgres (new project)
Auth	Supabase Auth	Supabase Auth (same library)
Storage	Supabase buckets (images)	Supabase buckets (product photos)
Front end	apps/landing-page/index.html	apps/store/index.html
Payments	Paystack init/verify/webhook	Paystack init/verify/webhook (reuse)
Email	Resend via api/email.js	Resend via api/email.js (reuse)

What you keep almost verbatim from Pulsify

- **api/email.js** — the deliver() + Resend/SMTP fallback and HTML email helpers.
- **Paystack flow** — /checkout/init → Paystack → /checkout/confirm + webhook, with the verifyPaystackTx() 3× retry and ZAR/amount assertions.
- **SA-bounds & validation helpers** — reused for delivery address sanity if needed.
- **Supabase client init + RLS habits** — service key server-only, anon key in the browser.
- **vercel.json** rule — only the API route; never add a catch-all to index.html.

What is new for an e-commerce store

- Product catalogue with variants (size, colour for clothing; volume/type for hair).
- Shopping cart + checkout (multi-item, not single-ticket).
- Inventory / stock tracking per variant.
- Orders + fulfilment status (paid → packed → shipped → delivered).
- Delivery / shipping options and address capture.

3. Database schema (Supabase / Postgres)

Create these tables in the new Supabase project. RLS ON for all; public can SELECT active products, only the service role writes orders.

products

- id (text, PK) — e.g. 'tshirt-oversize-black'
- name, description, category ('clothing' | 'hair')
- subcategory ('tops', 'bottoms', 'shampoo', 'treatment'...)
- base_price (numeric, ZAR), currency default 'ZAR'
- images (text[] of Supabase Storage URLs)
- is_active (bool), hype/featured (bool), created_at

product_variants

- id (uuid, PK), product_id (FK → products.id)
- label ('S / Black', '500ml')
- size, colour, volume (nullable — clothing uses size/colour, hair uses volume/type)
- price_override (nullable — falls back to products.base_price)
- stock (int) — decremented on confirmed order
- sku (text, unique)

orders

- id (uuid, PK), order_ref (text, unique — 'ORD-...')
- user_id (nullable — guests allowed), buyer_name, buyer_email, buyer_phone
- items (jsonb — snapshot of variant_id, name, qty, unit_price)
- subtotal, shipping_fee, total (numeric ZAR)
- status ('pending'|'paid'|'packed'|'shipped'|'delivered'|'cancelled')
- paystack_ref, shipping_address (jsonb), created_at

order_items (optional — normalised alternative to items jsonb)

- id (uuid), order_id (FK), variant_id (FK), quantity, unit_price

Stock rule: decrement product_variants.stock only when an order flips to 'paid' (inside the verified webhook / confirm handler) — never on cart add. Mirror Pulsify's 'verify before you flip status' discipline.

4. Front-end pages (single-file SPA)

One `apps/store/index.html` with tab/section switching, exactly like Pulsify's `index.html`. Sections:

Section	Purpose
Home / Shop	Hero banner, featured products, category tiles (Clothing / Hair).
Product list	Grid filtered by category, subcategory, price, in-stock. Search box.
Product detail	Image gallery, variant picker (size/colour or volume), Add to Cart.
Cart	Line items, qty steppers, subtotal, 'Checkout' CTA. Persisted in <code>localStorage</code> .
Checkout	Name/email/phone + delivery address, shipping option, Paystack pay button.
Order confirmation	Order ref, summary, 'we emailed your receipt'.
Account / Orders	Login (Supabase Auth) to see past orders + status.
Admin	Separate <code>apps/admin/</code> — add/edit products, manage stock, fulfil orders.

UX patterns to lift from Pulsify

- Bottom nav bar (mobile-first) — Shop / Cart / Orders / Account.
- String-injection rendering of cards (never destroy live DOM nodes mid-render).
- Single `searchTimeout` debounce for the product search.
- `escapeHtml()` on all user/admin text rendered into the page.
- Image lazy-loading; validate image URLs before render.

Clothing vs Hair — one catalogue, two faces

- Same products table; 'category' splits the two storefront entrances.
- Clothing variants emphasise size + colour; hair variants emphasise volume + type (e.g. 'Curl Cream 250ml', 'Shampoo 500ml').
- Shared cart & checkout — a customer can buy a hoodie and a hair treatment in one order.

5. Payments, email & delivery

Paystack (reuse Pulsify code)

- Cart → **POST /checkout/init**: server recomputes the total from DB prices (never trust the client amount), creates a 'pending' order, returns Paystack auth URL.
- Customer pays on Paystack → redirect back to **/checkout/confirm?ref=...**
- Confirm + webhook both call **verifyPaystackTx(ref, expectedZAR)**: 3× retry, assert currency=='ZAR' and amount >= total*100, then flip order to 'paid' and decrement stock.
- Gate go-live behind a feature flag (like Pulsify's 'paystack_live') until your keys are live.

Email (reuse api/email.js)

- Order confirmation email with line items + order ref (new template, same layout()/card()/btn() helpers).
- Shipping-update email when admin marks an order 'shipped' (add tracking link).
- Resend primary, SMTP fallback — identical to Pulsify.

Delivery / shipping

- Start simple: flat-rate SA shipping fee + free over a threshold (e.g. free over R750).
- Capture shipping_address as jsonb; show it on the admin order card for packing.
- Later: integrate a courier (e.g. The Courier Guy / Pargo pickup points) via their API.

6. Phased build plan

Hand each phase to a fresh Claude Code session. Verify before moving on.

Phase	Deliverable	Verify
0. Setup	New GitHub repo, Vercel project, Supabase project. Env vars visible in Vercel; Black deploy loads on a vercel.app URL.	
1. Schema	Create products / variants / orders tables + RLS + seed 5 demo products.	Products visible in Supabase; anon can SELECT active products.
2. Catalogue	Shop + product list + product detail pages reading from Supabase.	Products render with images and variant pickers.
3. Cart	Add to cart, qty, localStorage persistence, cart page.	Cart survives refresh; totals correct.
4. Checkout	Address capture + Paystack init/confirm/webhook + order creation.	Test-mode payment creates a 'paid' order, stock drops.
5. Emails	Order confirmation + shipping emails via Resend.	Receipt email arrives after a test order.
6. Admin	Add/edit products, manage stock, fulfil orders.	Can create a product and mark an order shipped.
7. Polish	Mobile nav, search, featured rail, SEO meta, favicon, domain.	Lighthouse mobile pass; custom domain live.

7. Kickoff prompt for the new session

Paste this into a fresh Claude Code session pointed at the new repo:

```
I'm building a new e-commerce store for clothing AND hair products, reusing the Pulsify stack (Vercel + Supabase + vanilla JS single-page front end). Read this handover PDF's schema and phased plan. We are on Phase 0: set up the Supabase schema (products, product_variants, orders) with RLS, then scaffold apps/store/index.html with a Shop / Cart / Checkout flow. Reuse Pulsify's api/email.js and Paystack verify-before-confirm pattern. Start by creating the Supabase tables and seeding 5 demo products (3 clothing, 2 hair). Then ask me for the brand name, colours, and logo before building the UI.
```

8. Decisions still needed from you

- **Brand name** for the store (repo name, domain, email from-address).
- **Domain** — buy a new one or use a subdomain.
- **Visual identity** — colours, logo, fonts (I can mock these in Canva).
- **Shipping policy** — flat rate? free-over-threshold amount? courier later?
- **Initial catalogue** — a list of real products with prices and photos to seed.

Prepared for Thabiso Collins Engane • Clothing & Hair brand on the Pulsify stack • Reuse Supabase/Vercel/GitHub accounts, new project per service.